

Université Abdelmalek Essaadi
École Nationale des Sciences Appliquées - Tétouan

Cycle Ingénieur - Génie Informatique (GI1)
Année Universitaire : 2025/2026

Projet Théorie des Langages

Réalisé par :

Bouarguan ABDELLAH
Ben Yacoub NIZAR
El Ghazouani MAROUANE
Cherradi ILYASS
El Younoussi NAFISA

Encadré par :

Pr. AL ACHHAB MOHAMMED

Période de réalisation : 23 Février 2026 - 11 Avril 2026

Sommaire

1	Introduction Générale	4
2	Introduction de la Phase 1	5
3	Recherche Préliminaire, Outils et Étude de Projets Existants	5
3.1	Concepts Théoriques	5
3.2	Format DOT et Écosystème Graphviz	5
3.3	Manipulation Visuelle	6
3.4	Inspiration et Étude de Projets Existants	6
4	Environnement de Travail et Choix Techniques	6
4.1	Le Langage C	6
4.2	Git et Collaboration sur GitHub	6
4.3	Standardisation de la Compilation avec CMake	6
5	Organisation et Répartition du Travail	8
6	Conception et Implémentation de la Phase 1	8
6.1	Structures de données (Question 1 : <code>automate.h</code>)	8
6.2	Lecture et stockage (Question 2 : Parsing dans <code>parser.c</code>)	9
6.3	Moteur d’affichage (Question 3 : <code>display.c</code>)	12
6.4	Interface interactive (Question 4 : <code>main.c</code>)	13
7	Difficultés Rencontrées et Solutions	13
8	Conclusion de la Phase 1	13
9	Introduction de la Phase 2	14
10	Répartition du Travail	14
11	Conception et Implémentation	14
11.1	Génération d’un fichier DOT (Question 5)	14
11.2	Statistiques sur les transitions (Question 6)	15
11.3	Recherche d’états par étiquette (Question 7)	15
11.4	Test de reconnaissance d’un mot (Question 8)	15
11.5	Traitement par lots depuis un fichier <code>.txt</code> (Question 9)	16
12	Difficultés Rencontrées et Solutions	16
13	Conclusion de la Phase 2	16
14	Conception et Implémentation de la Phase 3	17
14.1	Concaténation de deux automates (Question 10)	17
14.2	Union de deux automates (Question 11)	18
14.3	Construction à partir d’une expression régulière (Question 12)	18

14.4 Suppression des ϵ -transitions (Question 13)	18
15 Représentations Graphiques des Tests	18
16 Organisation et Répartition du Travail	20
17 Introduction de la Phase 4	21
18 Recherche Préliminaire et Concepts Théoriques	21
19 Organisation et Répartition du Travail	21
20 Conception et Implémentation de la Phase 4	22
20.1 Produit (Intersection) de deux automates (Question 14)	22
20.2 Détermination d'un automate (Question 15)	22
20.3 Minimisation par Algorithme de Brzozowski (Question 16)	22
20.4 Génération automatique des fichiers DOT (Question 17)	22
20.5 Filtrage des mots acceptés (Question 18)	23
21 Difficultés Rencontrées et Solutions	23
22 Conclusion de la Phase 4	23
23 Introduction des Phases 5 et 6	24
24 Concepts Théoriques et Analyse Lexicale	24
25 Organisation et Répartition du Travail	24
26 Conception et Implémentation	25
26.1 Génération de la table des symboles (Partie 5)	25
26.2 Intégration du Pipeline Automatisé (Partie 6)	25
27 Difficultés Rencontrées et Solutions	25
28 Conclusion de cette dernière phase	25
29 Conclusion Générale	26

1 Introduction Générale

Dans le cadre du module de Théorie des Langages et Compilation, l'objectif global de ce projet est de développer un programme permettant de remplir une partie de la table des symboles à partir soit d'une expression régulière, soit d'un automate fini décrit sous forme d'un fichier dot. Ce rapport retrace l'évolution de notre projet et se divise en plusieurs parties correspondantes aux différentes phases de développement.

Partie 1 : Lecture et affichage d'un automate

Période de réalisation : 23 Février 2026 - 29 Février 2026

2 Introduction de la Phase 1

Ce premier volet détaille la "Partie 1 : Lecture et affichage d'un automate", première étape de notre travail. Il retrace notre démarche chronologique, de la recherche théorique à l'implémentation finale des structures de données, en passant par la mise en place d'un environnement de travail collaboratif robuste.

3 Recherche Préliminaire, Outils et Étude de Projets Existants

3.1 Concepts Théoriques

D'un point de vue formel, et comme défini dans le support de cours de Théorie des Langages, un automate fini (ou machine à états finis) est un modèle mathématique de calcul. Il est précisément caractérisé par un 5-uplet (Q, A, δ, q_0, F) où :

- Q représente l'ensemble des états (qui est fini dans le cas d'un automate fini) ;
- A désigne l'alphabet de l'entrée ;
- δ est la fonction de transition, définie formellement comme $\delta : Q \times A \rightarrow Q$;
- $q_0 \in Q$ est l'état initial, sachant qu'un automate peut éventuellement posséder plusieurs états initiaux ($q_0 \subseteq Q$) ;
- $F \subseteq Q$ constitue l'ensemble des états d'acceptation (ou états finaux).

En informatique, ces modèles sont fondamentaux dans la conception de compilateurs, l'analyse lexicale, et la vérification de systèmes.

3.2 Format DOT et Écosystème Graphviz

Pour décrire la structure de nos automates, nous avons opté pour le format `.dot`, conformément aux spécifications du projet. Il s'agit d'un langage de description de graphes en texte brut issu de l'écosystème open-source **Graphviz**. Ce format intuitif modélise naturellement les états sous forme de nœuds et les transitions sous forme d'arêtes orientées (*edges*).

3.3 Manipulation Visuelle

Afin de vérifier la validité de notre parsing, il était nécessaire de visualiser graphiquement les automates. Sur environnement Linux, nous avons utilisé l'utilitaire de commande `dot` fourni par Graphviz pour générer des images (PNG ou PDF) à partir de nos fichiers textes. La commande employée était typiquement : `dot -Tpng automate.dot -o automate.png`

Pour faciliter le flux de travail des membres de l'équipe opérant sous environnement Windows, des convertisseurs Graphviz en ligne, tels que *GraphvizOnline*, ont été utilisés comme alternative efficace sans nécessiter d'installation locale complexe.

3.4 Inspiration et Étude de Projets Existants

Avant d'entamer la phase d'implémentation, une recherche a été menée sur des plateformes de partage de code telles que **GitHub**. L'analyse de projets open-source similaires (notamment des parsers de graphes développés en C et des manipulateurs d'expressions régulières) nous a permis d'identifier les meilleures pratiques de l'industrie. Ces recherches nous ont particulièrement inspirés sur la gestion dynamique de la mémoire et les design patterns d'analyse syntaxique pour le traitement de fichiers texte de manière robuste.

4 Environnement de Travail et Choix Techniques

4.1 Le Langage C

Le langage **C** a été retenu pour le développement de ce projet. Ce choix se justifie d'une part par son adéquation naturelle avec les concepts bas niveau de la théorie des langages (développement d'analyseurs lexicaux et syntaxiques). D'autre part, le C offre un contrôle fin sur la gestion de la mémoire via les pointeurs, une caractéristique indispensable pour optimiser la manipulation des tables de symboles et des graphes volumineux.

4.2 Git et Collaboration sur GitHub

Étant une équipe de 5 membres, l'utilisation d'un système de contrôle de version distribué tel que **Git** (hébergé sur GitHub) était impérative. L'intégralité du code source de notre projet ainsi que l'historique de nos contributions sont accessibles sur notre dépôt public : https://github.com/AbdellahBouarguan/projet_theorie_langages.

La collaboration s'est orchestrée autour de la stratégie du *Feature Branching* : chaque développeur travaillait sur une branche isolée pour sa tâche spécifique avant de soumettre une *Pull Request*. La protection de la branche principale (**main**) a été instaurée pour empêcher l'écrasement involontaire du code et s'assurer que seules des implémentations testées y soient fusionnées.

4.3 Standardisation de la Compilation avec CMake

Initialement, la compilation du projet reposait sur un simple **Makefile**. Cependant, une problématique de portabilité s'est rapidement imposée au sein de notre équipe hétérogène (utilisation de Linux, couplée à des systèmes d'exploitation Windows). Les commandes de shell Unix appelées dans le fichier **Make** n'étaient pas reconnues nativement sous Windows.

Pour résoudre ce goulet d'étranglement, nous avons migré vers **CMake**. Cet outil génère un processus de compilation agnostique vis-à-vis du système d'exploitation, créant

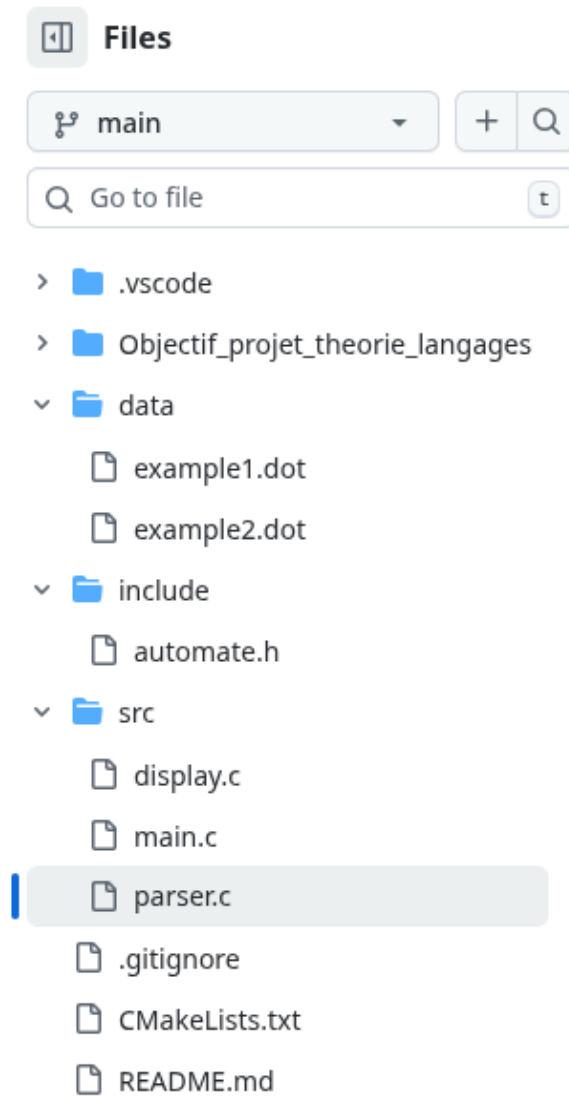


FIGURE 1 – Structure de notre dépôt public sur GitHub

Default	
Branch	Updated
main	yesterday
Active branches	
Branch	Updated
feature-nafissa-affichageAndLastC...	8 minutes ago
feature-nizar-display	yesterday
feature-ilyass-parser	yesterday
feature-marouane-structures	yesterday

FIGURE 2 – Illustration de notre stratégie de Feature Branching

des **Makefiles** sous Linux et des solutions Visual Studio ou MinGW sous Windows, uniformisant ainsi notre chaîne de build multiplateforme.

5 Organisation et Répartition du Travail

La réussite de cette première phase repose sur une répartition stricte et coordonnée des responsabilités entre les membres de l'équipe :

- **Abdellah** : Chargé de l'initialisation de l'architecture des répertoires et de la configuration de l'infrastructure de base (Git et CMake). Il supervise le bon fonctionnement du projet et la coordination de l'équipe, gère la chaîne de compilation et l'exécution du programme, et assure la rédaction documentaire garantissant la traçabilité des choix techniques.
- **Nizar** : Responsable de la traduction des automates finis vus en cours vers le format DOT à l'aide de l'outil en ligne **GraphvizOnline** (génération des fichiers `data/example1.dot` et `data/example2.dot`). Il a également pris en charge la modélisation mathématique en structures de données logicielles (conception du module `automate.h`) et a contribué au développement du moteur d'affichage (`display.c`).
- **Marouane** : En charge du développement du moteur d'analyse syntaxique permettant la lecture et l'extraction de données depuis les fichiers DOT (`parser.c`). Il a également contribué à la conception de la structure de données (`automate.h`) et a initialisé l'intégration de cet automate au sein du *parser*.
- **Ilyas** : Développeur du module d'affichage console, s'assurant que la machinerie interne soit exposée lisiblement à l'écran (`display.c`). Il a aussi activement participé au développement du module d'analyse syntaxique (`parser.c`).
- **Nafisa** : Responsable de l'intégration globale des modules et conceptrice de l'interface utilisateur interactive (le menu principal dans `main.c`).

6 Conception et Implémentation de la Phase 1

6.1 Structures de données (Question 1 : `automate.h`)

Afin de répondre au premier objectif du cahier des charges de la **Question 1** demandant de proposer des structures de données permettant l'implémentation d'un automate fini, nous avons défini des structures en C capables de stocker les états, les transitions, l'alphabet, ainsi que la nature de chaque état (initial, final).

La structure **Transition** modélise un arc entre deux états, étiqueté par un symbole de l'alphabet :

```
1
2 // Structure pour une transition
3 typedef struct {
4     int from_etat;
5     int to_etat;
6     char label[MAX_LABEL_LEN];
```

Listing 1 – Structure **Transition** (extraite de `automate.h`)

La structure **Automaton** regroupe toutes les informations pertinentes pour définir formellement l'automate (états, transitions, alphabet, booléens pour la nature des états) :


```

1 #define MAX_ETATS 100
2 #define MAX_TRANSITIONS 500
3 #define MAX_ALPHABET 50
4 #define MAX_LABEL_LEN 64
5
6 // Structure pour l'automate
7 typedef struct {
8     int num_etats;
9     int etats[MAX_ETATS]; // Liste des identifiants (ex: 0, 1, 2)
10    bool is_final[MAX_ETATS]; // Indique si l'état est final
11    bool is_initial[MAX_ETATS]; // Indique si l'état est initial
12
13    int num_transitions;
14    Transition transitions[MAX_TRANSITIONS];
15
16    int num_alphabet;
17    char alphabet[MAX_ALPHABET][MAX_LABEL_LEN];

```

Listing 2 – Structure Automaton (extraite de automate.h)

6.2 Lecture et stockage (Question 2 : Parsing dans parser.c)

Pour la **Question 2**, l'objectif consistait à ajouter une fonction permettant de lire et stocker un automate fini à partir d'un fichier `.dot`. L'un des défis majeurs fut l'extraction fiable des informations pertinentes de ce format texte.

Nous avons implémenté la fonction `load_automaton_from_dot`, qui constitue le cœur de notre analyseur syntaxique (*parser*). Son rôle est de parcourir le fichier texte et de traduire les descriptions graphiques en données exploitables en mémoire. Afin de la rendre robuste et de contourner les spécificités du format DOT, la fonction opère selon les étapes séquentielles suivantes :

1. **Ouverture sécurisée du fichier** : Le fichier est ouvert en mode lecture ("r"). Si le fichier est introuvable ou illisible, le programme signale une erreur dans la console et interrompt le chargement proprement, évitant ainsi une erreur de segmentation.
2. **Filtrage ligne par ligne** : Le fichier est lu ligne après ligne via la fonction `fgets`. Pour optimiser le traitement, le programme ignore les lignes de décoration globale et se concentre exclusivement sur les lignes définissant des relations, identifiables par la présence de la chaîne d'arête `->`.
3. **Détection de l'état initial (Astuce du nœud fantôme)** : Dans les conventions de tracé Graphviz, un état initial est souvent matérialisé par une flèche provenant d'un nœud invisible ou vide (par exemple : `" -> 0`). Notre algorithme intercepte ce motif vide `"`, le remplace dynamiquement en mémoire par un identifiant lisible (`"IN"`), ce qui permet à la fonction `sscanf` de l'analyser. L'état pointé par cette flèche voit alors son attribut `is_initial` basculer à `true`.
4. **Détection des états finaux** : De manière similaire, notre code gère une autre convention où un état final pointe vers un nœud d'arrêt (par exemple : `3 -> fin_node`). Si la destination de la flèche commence par les lettres `"fin"`, l'état d'origine est enregistré comme étant un état acceptant (`is_final = true`).
5. **Extraction des transitions régulières et construction de l'alphabet** : Pour les véritables transitions entre deux états de l'automate :

- Le programme isole les identifiants de l'état de départ et de l'état d'arrivée, en prenant soin de nettoyer les éventuels caractères parasites de syntaxe (tels que les crochets d'attributs [ou les points-virgules de fin de ligne ;).
- Il recherche ensuite l'attribut `label=` pour en extraire le symbole de transition, en gérant de manière sécurisée la lecture entre les guillemets.
- Ces informations sont structurées sous forme d'une `Transition` puis ajoutées au tableau `transitions` de notre automate.
- *Gestion de l'alphabet* : Si l'étiquette extraite n'est pas le mot vide (représenté ici par la chaîne "epsilon"), elle est envoyée à la fonction `add_to_alphabet` pour enrichir dynamiquement l'alphabet global de l'automate sans créer de doublons.

6. **Sécurité par défaut (*Fallback initial*)** : Une fois le fichier entièrement lu et refermé, le programme effectue une ultime vérification. Si le formatage du fichier DOT ne précisait explicitement aucun état initial, l'algorithme affecte cette propriété par défaut au premier nœud identifié (indice 0), garantissant ainsi que l'automate reste valide et manipulable pour les opérations futures.

```

1 bool load_automaton_from_dot(Automaton *automate, const char *filename)
2 {
3     FILE *file = fopen(filename, "r");
4     if (!file)
5     {
6         printf("Erreur: Impossible d'ouvrir le fichier %s\n", filename);
7         return false;
8     }
9
10    char line[256];
11
12    while (fgets(line, sizeof(line), file))
13    {
14        char *arrow_pos = strstr(line, "->");
15
16        // Nous nous intéressons uniquement aux lignes contenant des
17        // transitions
18        if (arrow_pos)
19        {
20            char from_str[64] = {0};
21            char to_str[64] = {0};
22            char label_str[64] = "epsilon";
23
24            // Solution pour les nœuds avec chaîne vide (ex: "" -> 0)
25            // Remplacer "" par un identifiant temporaire "IN" afin que sscanf
26            // puisse l'analyser comme un seul token
27            char temp_line[256];
28            strcpy(temp_line, line);
29            char *empty_node = strstr(temp_line, "\\\"");
30            if (empty_node && empty_node < (temp_line + (arrow_pos - line)))
31            {
32                empty_node[0] = 'I';
33                empty_node[1] = 'N'; // Transforme "" en IN
34            }
35
36            // Extraire la source et la destination
37            if (sscanf(temp_line, " %s -> %s", from_str, to_str) != 2)
38                continue;
39        }
40    }
41}

```

```

38 // Supprimer les crochets ou points-virgules à la fin de la
    destination
39 char *bracket = strchr(to_str, '[');
40 if (bracket)
41     *bracket = '\0';
42 char *semi = strchr(to_str, ';');
43 if (semi)
44     *semi = '\0';
45
46 // 1. Détection de l'état initial ("IN" -> État)
47 if (strcmp(from_str, "IN") == 0)
48 {
49     int to_id = parse_etat_id(to_str);
50     if (to_id != -1)
51     {
52         int idx = get_or_create_etat_idx(automate, to_id);
53         automate->is_initial[idx] = true;
54     }
55     continue;
56 }
57
58 // 2. Détection de l'état final (État -> "fin*")
59 if (strncmp(to_str, "fin", 3) == 0)
60 {
61     int from_id = parse_etat_id(from_str);
62     if (from_id != -1)
63     {
64         int idx = get_or_create_etat_idx(automate, from_id);
65         automate->is_final[idx] = true;
66     }
67     continue;
68 }
69
70 // 3. Traitement des transitions normales
71 int from_id = parse_etat_id(from_str);
72 int to_id = parse_etat_id(to_str);
73
74 if (from_id != -1 && to_id != -1)
75 {
76     char *label_pos = strstr(line, "label");
77     if (label_pos)
78     {
79         char *quote1 = strchr(label_pos, '"');
80         if (quote1)
81         {
82             char *quote2 = strchr(quote1 + 1, '"');
83             if (quote2)
84             {
85                 strncpy(label_str, quote1 + 1, quote2 - quote1 - 1);
86                 label_str[quote2 - quote1 - 1] = '\0';
87             }
88         }
89         else
90         {
91             sscanf(label_pos, "label=%63[~] \t;]", label_str);
92         }
93     }
94 }

```

```

95     int from_idx = get_or_create_etat_idx(automate, from_id);
96     int to_idx = get_or_create_etat_idx(automate, to_id);
97
98     if (automate->num_transitions < MAX_TRANSITIONS)
99     {
100         Transition *t = &automate->transitions[automate->
num_transitions++];
101         t->from_etat = automate->etats[from_idx];
102         t->to_etat = automate->etats[to_idx];
103         strcpy(t->label, label_str);
104
105         if (strcmp(label_str, "epsilon") != 0)
106         {
107             add_to_alphabet(automate, label_str);
108         }
109     }
110 }
111 }
112 }
113 // Par défaut, considérer le premier état (index 0) comme initial
114 // si aucun état initial n'est explicitement défini via "" -> X
115 bool has_initial = false;
116 for (int i = 0; i < automate->num_etats; i++)
117 {
118     if (automate->is_initial[i])
119     {
120         has_initial = true;
121         break;
122     }
123 }
124 if (!has_initial && automate->num_etats > 0)
125 {
126     automate->is_initial[0] = true;
127 }
128
129 fclose(file);
130 return true;
131 }

```

Listing 3 – Logique de la fonction `load_automaton_from_dot` (fichier `parser.c`)

6.3 Moteur d’affichage (Question 3 : `display.c`)

Une fois l’automate chargé, la **Question 3** du cahier des charges exigeait d’ajouter une fonction permettant d’affichage les informations d’un automate passé en paramètre la liste des états, la liste des transitions, l’alphabet, les états finaux, les états initiaux. Ainsi, la fonction `display_automaton` a été développée.

Son rôle est de parcourir la structure en mémoire et d’afficher de façon lisible dans la console :

- La liste de tous les états récupérés.
- La liste restreinte des états initiaux et finaux (états acceptants).
- L’alphabet complet déduit des étiquettes des transitions.
- La liste complète des transitions formatée (ex : $\delta(q_0, a) = q_1$).

6.4 Interface interactive (Question 4 : `main.c`)

Pour répondre à la **Question 4** et orchestrer ces différentes fonctionnalités, nous devions ajouter un menu à notre programme. Ce menu a été intégré au fichier `main.c` et propose les options suivantes de manière cyclique :

1. **Charger un automate** : L'utilisateur saisit le chemin du fichier `.dot`.
2. **Afficher les informations** : Invoque la fonction `display_automaton`.
3. **Quitter** : Libère la mémoire et termine le programme proprement.

7 Difficultés Rencontrées et Solutions

Le développement de cette première itération n'a pas été sans obstacles. Nous soulignons notamment trois problématiques majeures résolues par l'équipe :

1. **Manipulation ardue des chaînes de caractères en C** : L'extraction fiable des labels de transition encadrés par des guillemets dans les fichiers DOT a nécessité des manipulations de pointeurs complexes et une forte vigilance quant aux débordements de tampons (buffers overflow), le C ne possédant pas de regex natives.
2. **Courbe d'apprentissage de Git** : Dans les premiers jours, la synchronisation du travail a été parasitée par des conflits de fusion (*merge conflicts*). Cela a imposé un temps d'adaptation et des sessions de résolution de conflits didactiques entre les membres.
3. **Incompatibilité OS pour la compilation** : La rigidité de notre premier `Makefile` n'a pas survécu à l'environnement Windows d'une partie de l'équipe, paralysant momentanément la compilation avant que nous n'adoptions l'approche *cross-platform* de CMake.
4. **Lecture des chemins de fichiers sous Windows** : Nous avons rencontré un problème lors de la saisie des chemins de fichiers avec la fonction `scanf("%255s", filename)`. Sous Windows, les chemins contiennent souvent des espaces et des antislashs qui faisaient échouer la lecture simple par cette fonction, nécessitant des ajustements pour capturer correctement la chaîne de caractères.

8 Conclusion de la Phase 1

En conclusion pour cette première semaine de développement, les bases techniques et organisationnelles du projet ont été solidement établies. La modélisation mathématique de l'automate fini a été transposée efficacement en structures C opérationnelles. Le système est désormais capable de dialoguer avec des fichiers standards (DOT) et de restituer l'information extraite, validant ainsi la totalité de la Partie 1 et ouvrant la voie à des manipulations plus complexes.

Partie 2 : Manipulation d'un automate

Période de réalisation : 2 Mars 2026 - 8 Mars 2026

9 Introduction de la Phase 2

Cette deuxième partie du projet porte sur la manipulation avancée des automates finis en validant les acquis théoriques du cours. Nous avons étendu notre programme pour inclure la génération de fichiers au format DOT, l'analyse des propriétés du graphe (transitions entrantes et sortantes), et l'évaluation de la reconnaissance de mots (simulation d'AFN). Ce travail a été intégré au menu interactif principal du programme.

10 Répartition du Travail

Tout comme la première phase, la répartition des tâches a été essentielle pour la réussite de cette deuxième étape :

- **Nizar et Nafisa** : Ont effectué une recherche initiale exhaustive sur les problématiques soulevées par les **Questions 5 à 9** afin de familiariser l'équipe avec les objectifs à atteindre. Ils se sont chargés d'expliquer les notions théoriques issues du cours du professeur et d'identifier la démarche algorithmique requise pour le traitement de chaque question.
- **Ilyas** : S'est chargé de la réalisation technique de la **Question 5**, concevant et implémentant l'outil interactif de génération de fichiers DOT au sein du module `generate.c`.
- **Abdellah et Marouane** : Ont collaboré sur l'implémentation algorithmique de la manipulation avancée des automates et de la simulation de reconnaissance de mots, en couvrant l'intégralité du développement pour les **Questions 6 à 9**.

11 Conception et Implémentation

11.1 Génération d'un fichier DOT (Question 5)

Pour répondre à la **Question 5**, nous avons implémenté la fonction `generate_dot` dans le fichier `generate.c`. Cette fonction permet de créer manuellement un automate de manière interactive depuis la console et de le sauvegarder dans un fichier `.dot`. Elle s'assure de respecter les conventions de nommage et notations du cours. Notamment, elle

utilise un nœud invisible "" pour pointer vers les états initiaux, et crée des arcs vers des états finalX pour représenter graphiquement les états d'acceptation.

```

1      fprintf(f, "%d -> %d [label=\"%s\"];\\n", a->transitions[i].from_etat
2      ,
3          a->transitions[i].to_etat, a->transitions[i].label);
4
5      fprintf(f, "node [shape=point];\\n");
6      for (int i = 0; i < a->num_etats; i++)
7          if (a->is_final[i])
8              fprintf(f, "%d -> final%d;\\n", a->etats[i], a->etats[i]);
9
10     fprintf(f, "}\\n");
11     fclose(f);
12     printf("Automate exporté avec succès dans '%s'.\\n", filename);

```

Listing 4 – Extrait de la génération DOT (fichier generate.c)

11.2 Statistiques sur les transitions (Question 6)

La **Question 6** demandait d'identifier l'état possédant le plus grand nombre de transitions entrantes et sortantes. Dans `manipulation.c`, la fonction `afficher_etat_max_transitions` calcule ces degrés pour chaque nœud en parcourant le tableau global des transitions de l'automate. Des compteurs locaux sont maintenus pour chaque état (`in_count` et `out_count`). Le programme détermine ensuite le maximum global et l'affiche à l'utilisateur.

11.3 Recherche d'états par étiquette (Question 7)

Pour la **Question 7**, l'objectif était d'afficher les états ayant au moins une transition sortante étiquetée par un symbole précis (passé en paramètre).

La fonction `afficher_etats_transition_label` parcourt les transitions de l'automate et compare l'étiquette demandée avec celle de chaque transition via `strcmp`. Les états correspondants sont extraits et affichés sans doublons, grâce à un tableau de marquage booléen `printed`.

11.4 Test de reconnaissance d'un mot (Question 8)

La vérification de mots (**Question 8**) constitue une partie essentielle. Comme un automate peut être non déterministe (AFN) et comporter des transitions ϵ (epsilon), la simple itération linéaire sur le mot ne suffit pas. Nous avons implémenté un algorithme de recherche en profondeur récursif (backtracking) dans la fonction interne `match_recursive`.

- L'algorithme avance dans le graphe de l'automate en consommant les lettres du mot.
- Les transitions ϵ sont franchies sans consommer de lettre. Pour éviter les boucles infinies de transitions ϵ , nous maintenons un tableau `visited_eps` des états déjà visités lors de l'évaluation du même caractère.
- Si toute la chaîne est consommée et que nous nous trouvons dans un état final (ou qu'un état final est atteignable via des transitions ϵ), le mot est accepté.

La fonction publique `tester_mot` initialise cette recherche à partir de tous les états initiaux. L'interaction avec l'utilisateur (saisie depuis l'invite de commande) est gérée par la méthode `lire_et_tester_mot`.

11.5 Traitement par lots depuis un fichier .txt (Question 9)

Enfin, pour la **Question 9**, la fonction `filtrer_mots_fichier` permet de vérifier automatiquement une série de mots lus depuis un fichier texte (les mots étant séparés par des espaces ou retours à la ligne). Pour chaque mot extrait par `fscanf`, la fonction invoque `tester_mot`. Si l'automate reconnaît le mot, ce dernier est écrit dans un fichier de sortie `MotsAccepter.txt`. À l'issue du traitement, le programme affiche un résumé statistique du nombre total de mots lus et du nombre de mots acceptés.

12 Difficultés Rencontrées et Solutions

Lors de cette deuxième phase de développement, l'équipe a fait face à de nouveaux défis techniques, principalement liés à la manipulation avancée des graphes :

1. **Mise en œuvre du Backtracking avec les transitions ϵ** : L'une des plus grandes difficultés a été la modélisation mathématique du non-déterminisme, en particulier les cycles infinis créés par les transitions epsilon (ϵ). *Solution* : Nous avons introduit un tableau booléen `visited_eps` dans notre algorithme récursif (`match_recursive`) pour mémoriser les états déjà traversés lors de l'évaluation d'une lettre, cassant ainsi les boucles infinies.
2. **Gestion des correspondances exactes de mots** : Lors de la vérification, il était parfois complexe de distinguer la fin d'un mot valide d'une correspondance partielle. *Solution* : La condition d'acceptation a été strictement définie : l'algorithme doit consommer l'intégralité de la chaîne de caractères (`pos == strlen(mot)`) ET se trouver dans un état final (ou pouvoir en atteindre un via epsilon).
3. **Création manuelle d'automates via la console** : La génération d'un fichier DOT valide (**Question 5**) via l'interface console introduisait des risques d'erreurs de saisie utilisateur risquant de corrompre le fichier généré. *Solution* : Le flux d'interaction de `generate.c` a été conçu de manière séquentielle et stricte, forçant d'abord la définition de l'architecture nodale (états normaux, initiaux, finaux) avant de lier les relations (nombre de transitions, états de départ/arrivée et labels).

13 Conclusion de la Phase 2

Les objectifs de la Partie 2 ont été atteints avec succès. Le système modélise fidèlement le comportement d'un AFN grâce à son algorithme de parcours incluant les ϵ -transitions, permettant ainsi de manipuler pleinement l'automate et de tester la reconnaissance de mots. Couplé aux fonctionnalités de parsing de la Phase 1, le programme final est complètement opérationnel et offre une gestion intégrale des automates finis.

Partie 3 : Opérations sur les automates

Période de réalisation : 9 Mars 2026 - 15 Mars 2026

14 Conception et Implémentation de la Phase 3

Cette phase se concentre sur les opérations algébriques et les transformations structurelles des automates, en s'appuyant sur les bases de données déjà établies.

14.1 Concaténation de deux automates (Question 10)

La **Question 10** demandait d'ajouter une fonction permettant de retourner la concaténation de deux automates passés en paramètres. Nous avons implémenté la fonction `concatener_automates`. La logique consiste à :

- Fusionner les ensembles d'états des deux automates en utilisant un décalage (*offset*) pour éviter les collisions d'identifiants.
- Relier chaque état final du premier automate aux états initiaux du second par des transitions ϵ (epsilon).
- Définir les états finaux du second automate comme les seuls états finaux de l'automate résultant.

```
1 Automaton *concatener_automates(const Automaton *a1, const Automaton *a2
  ) {
2     Automaton *res = create_automaton();
3     copy_and_offset_states(res, a1, 0);
4     int offset = get_max_id(a1) + 1;
5     copy_and_offset_states(res, a2, offset);
6
7     for (int i = 0; i < a1->num_etats; i++) {
8         if (a1->is_final[i]) {
9             res->is_final[i] = false;
10            for (int j = 0; j < a2->num_etats; j++) {
11                if (a2->is_initial[j]) {
12                    add_transition(res, res->etats[i], res->etats[a1->num_etats+j
13                        ], "epsilon");
14                }
15            }
16        }
17    }
18    return res;
```

14.2 Union de deux automates (Question 11)

Pour répondre à la **Question 11**, la fonction `union_automates` crée un nouvel automate qui accepte l'union des langages des deux automates sources. La construction repose sur l'ajout d'un nouvel état initial "super-source" relié par des transitions ϵ aux anciens états initiaux des deux automates. Un état final commun est également ajouté pour centraliser l'acceptation.

14.3 Construction à partir d'une expression régulière (Question 12)

La **Question 12** concernait la construction d'un automate à partir d'une expression régulière. Nous avons implémenté l'algorithme de **Thompson** via la fonction `regex_to_nfa`. Cet algorithme transforme récursivement les opérateurs de l'expression (union $|$, concaténation, et étoile de Kleene $*$) en fragments d'automates reliés par des transitions ϵ . L'analyse syntaxique s'appuie sur l'algorithme *Shunting-Yard* pour gérer les priorités d'opérateurs.

14.4 Suppression des ϵ -transitions (Question 13)

Enfin, pour la **Question 13**, la fonction `supprimer_epsilon_transitions` transforme un AFN- ϵ en un AFN classique. L'algorithme utilise le concept de **fermeture-epsilon** (ϵ -closure). Pour chaque état, nous calculons l'ensemble des états atteignables via ϵ uniquement par l'algorithme de Floyd-Warshall. Pour chaque transition $q_i \xrightarrow{a} q_j$, si q_j peut atteindre un état q_k via une ou plusieurs transitions ϵ , une nouvelle transition directe $q_i \xrightarrow{a} q_k$ est créée, puis les ϵ physiques sont supprimés.

15 Représentations Graphiques des Tests

Pour valider nos implémentations, nous avons généré des fichiers `.dot` à partir des résultats de nos opérations.

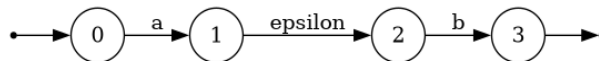


FIGURE 3 – Représentation graphique de la concaténation (testé sur a.dot et b.dot)

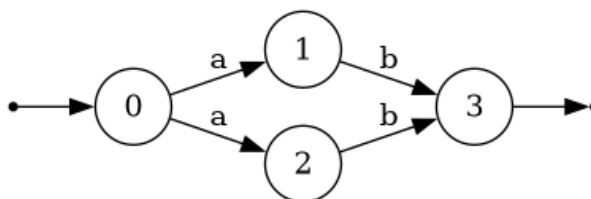


FIGURE 4 – Automate après suppression des transitions epsilon

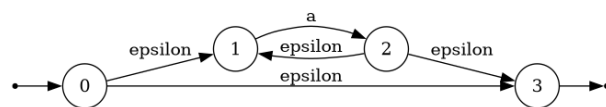


FIGURE 5 – Automate généré à partir d’une expression régulière (Thompson)

16 Organisation et Répartition du Travail

Pour cette phase finale, l'équipe a adopté une structure de développement en couches :

- **Abdellah** : Architecture des opérations complexes et implémentation de l'algorithme de Thompson (regex vers NFA) .
- **Nizar** : Conception de la logique de réindexation (*offsetting*) et des outils de fusion et implémentation de l'algorithme Shunting-Yard.
- **Marouane** et **Ilyass** : Développement et optimisation du module de fermeture-epsilon et de la structure de backtracking.
- **Nafisa** : Maintenance globale, tests d'intégration et mise à jour de l'interface utilisateur pour inclure les nouvelles opérations.

Partie 4 : Optimisation des automates

Période de réalisation : 29 Mars 2026 - 4 Avril 2026

17 Introduction de la Phase 4

Cette phase finale porte sur l'optimisation mathématique des automates finis (produit analytique, détermination et minimisation), afin de réduire leur taille en mémoire et d'améliorer leur efficacité de traitement tout en préservant stricto sensu le langage reconnu.

18 Recherche Préliminaire et Concepts Théoriques

Avant l'implémentation, nous nous sommes appuyés sur les définitions formelles du cours :

- **Produit d'automates** : Permet de construire un automate reconnaissant l'intersection des langages $L(A)$ et $L(B)$. Les états du nouvel automate sont définis par le produit cartésien des états, soit des couples (q_1, q_2) .
- **Détermination** : Transformation d'un AFN en AFD équivalent via la méthode de construction par sous-ensembles (calcul des fermetures epsilon et des états accessibles).
- **Minimisation (Algorithme de Brzozowski)** : Méthode algébrique élégante reposant sur la transposition et la détermination successive : $Det(Trans(Det(Trans(A))))$.
- **Transposition** : Inversion du sens de l'ensemble des transitions et échange des rôles entre états initiaux et finaux d'un automate.

19 Organisation et Répartition du Travail

Pour mener à bien cette dernière itération d'optimisation complexe, l'équipe s'est de nouveau divisée stratégiquement :

- **Abdellah** : En charge du module de filtrage des mots lus depuis le fichier texte, des tests d'intégration complets, et de la mise à jour finale de l'interface utilisateur.
- **Nizar** : Développement exclusif du moteur de minimisation (Algorithme de Brzozowski) et de la logique de transposition.
- **Marouane** : Modélisation mathématique et implémentation fine du produit cartésien d'automates.

- **Ilyass** : Ingénierie sur le moteur `.dot` pour la génération automatique du *pipeline* visuel (initial, déterministe, minimal).
- **Nafisa** : Architecture logicielle globale, développement de la délicate phase de déterminisation (manipulation des sous-ensembles) et coordination de l'équipe lors de la fusion finale.

20 Conception et Implémentation de la Phase 4

20.1 Produit (Intersection) de deux automates (Question 14)

Pour la **Question 14**, nous avons développé la fonction `produit_automates`. Son algorithme utilise un parcours en largeur (BFS) à l'aide d'une file d'attente (queue) pour générer récursivement les états valides atteignables. Pour garantir l'unicité des nouveaux états couplés créés à partir des paires (q_1, q_2) , nous appliquons une technique de codage mathématique condensée : `dest = s1 * base + s2`, où la variable `base` correspond à l'identifiant maximum des états du second automate incrémenté de un.

20.2 Déterminisation d'un automate (Question 15)

Pour la **Question 15**, notre fonction `determiniser` implémente la fameuse *powerset construction*. Chaque état du nouvel AFD généré correspond à un ensemble d'états de l'AFN de départ. Un système d'indexation via un tableau bidimensionnel (`sets[MAX_ETATS][MAX_ETATS]`) est utilisé pour mémoriser les sous-ensembles construits, vérifier rigoureusement s'ils ont déjà été rencontrés avant d'ajouter une nouvelle transition et ainsi éviter les redondances ou boucles illogiques.

20.3 Minimisation par Algorithme de Brzozowski (Question 16)

Pour la **Question 16**, l'objectif est d'obtenir l'automate DFA le plus compact reconnaissant le langage global. Implémentée dans `minimiser_brzozowski`, la méthode s'articule ainsi :

```

1 Automaton *minimiser_brzozowski(const Automaton *a) {
2     Automaton *r1 = transposer_automaton(a);
3     Automaton *d1 = determiniser(r1);
4     Automaton *r2 = transposer_automaton(d1);
5     Automaton *d2 = determiniser(r2);
6     // (Puis libération de la mémoire allouée par r1, d1, r2...)
7     return d2;
8 }

```

Listing 6 – Principe interne de l'algorithme de Brzozowski (operations.c)

La fonction de transposition (`transposer_automaton`) associée intervertit le sens de toutes les flèches du graph en s'appuyant sur les tableaux de booléens `is_initial` et `is_final`.

20.4 Génération automatique des fichiers DOT (Question 17)

Ciblant la **Question 17**, notre outil `generer_dot_pipeline` automatise un *pipeline* complet de rendu graphique. Lorsqu'on lui fournit un automate original en mémoire, il enchaîne les fonctions de transformations requises pour générer instantanément les trois sous-fichiers :

- `xyz_initial.dot`
- `xyz_deterministe.dot`
- `xyz_minimal.dot`

20.5 Filtrage des mots acceptés (Question 18)

Enfin, la **Question 18** valide fonctionnellement l'automate le plus minime à travers la routine `afficher_mots_acceptes`. Le parser C itère sur un fichier d'essai texte, ligne par ligne. Via l'implémentation `mot_accepte`, le programme simule le parcours séquentiel au sein du graph minimal pour chaque mot lue, en affichant élégamment **ACCEPTÉ** ou un **REJETÉ**.

21 Difficultés Rencontrées et Solutions

Comme pour les étapes mathématiques précédentes, divers obstacles ont ralenti l'équipe :

1. **Explosion Combinatoire** : La complexité exponentielle due à la modélisation en mémoire des sous-ensembles de la détermination nous a causé des soucis mémoires. Un système itératif par tableau en C a été préféré à la récursion pure.
2. **Filtrage Epsilon** : Les transitions Epsilon ont posé de sérieux problèmes d'exécution en provoquant des boucles lors des calculs de fermetures. Il a fallu s'assurer que notre système les évite pour la détermination DFA de cette 4ème étape.
3. **Fusion Minutieuse post-Transposition** : Le respect des multiples états initiaux après la première inversion a nécessité un traitement particulier dans la méthode de Brzowski afin de ne pas fausser la définition de l'AFD intermédiaire.

22 Conclusion de la Phase 4

À l'issue de cette quatrième phase de développement, le pipeline de traitement s'avère totalement opérationnel. L'équipe est en mesure d'analyser, de combiner, puis d'optimiser radicalement la taille de structure des automates finis d'entrée complexes sans perdre la validité des grammaires de langages qu'ils représentent.

Partie 5 et 6 : Table des Symboles et Intégration

Période de réalisation : 5 Avril 2026 - 11 Avril 2026

23 Introduction des Phases 5 et 6

Les deux ultimes phases couronnent l'aboutissement du projet. La **Partie 5** modélise l'étape fondamentale de l'analyse lexicale d'un compilateur : la création d'une Table des Symboles. La **Partie 6** consiste en une validation croisée globale du pipeline logiciel entier, transformant une simple expression régulière lue depuis un fichier en un analyseur lexical complet, automatisé et graphique.

24 Concepts Théoriques et Analyse Lexicale

- **Lexème (Token)** : Séquence de caractères formant une unité sémantique valide (reconnu par l'automate du langage).
- **Table des Symboles** : Structure de données vitale orientée dictionnaire permettant au compilateur ou l'interpréteur de retracer chaque variable/mot et de retenir son numéro de ligne pour une potentielle gestion future des erreurs syntaxiques.
- **Automate Canonique** : C'est le résultat direct de la minimisation de l'automate DFA. L'automate le plus minimal et le plus performant pour opérer l'analyse lexicale sans perte de temps.

25 Organisation et Répartition du Travail

Pour cette ligne droite finale, l'équipe s'est réunie afin de finaliser et lier les différents modules indépendants construits jusqu'ici :

- **Nafisa** et **Nizar** : Implémentation du système de *tokenisation* avec le suivi de numéros de lignes (découpage du fichier source) pour la Partie 5.
- **Abdellah** et **Marouane** : Conception de la fonction directrice de la Partie 6 branchant les séquences (Regex $\rightarrow$ NFA $\rightarrow$ DFA $\rightarrow$ Minimal $\rightarrow$ Table des Symboles).
- **Ilyass** : Finalisation des rapports visuels (`partie6_thompson.dot` et `partie6_canonique.dot`) et ajustements de l'interface console.

26 Conception et Implémentation

26.1 Génération de la table des symboles (Partie 5)

La fonction `generer_table_symboles` lit un fichier source complet (`fichier_txt`) ligne par ligne à travers la fonction `fgets`. Elle maintient prudemment un compteur d'avancement `num_ligne`. En C, nous accomplissons l'extraction lexicale en utilisant la fonction standard `strtok` qui découpe la ligne en termes d'espaces et sauts de lignes de façon itérative. Chaque lexème (*token*) extrait sert de mot d'alimentation pour notre simulateur d'automate interne (`mot_accepte`). Si validé, ce lexème ainsi que sa ligne courante, tombent dans une structure logicielle précise `Symbole table[MAX_SYMBOL]` qui s'édite visuellement sur la console du terminal.

26.2 Intégration du Pipeline Automatisé (Partie 6)

Pour relier les fondations de ce compilateur sémantique, la fonction intégratrice `traiter_partie_6` charge un fichier `regex.txt`. Dans un balai coordonné, le pipeline informatique développé lance l'action suivante :

1. Transformation de l'expression mathématique et construction récursive de l'automate formel (`regex_to_nfa`).
2. Impression de la vue de Thompson en DOT.
3. Purge des faiblesses (`supprimer_epsilon_transitions`) et déterminisation solide de l'Automate (`determiniser`).
4. Tassement du graph avec `minimiser_brzozowski` pour accoster sur un **Automate Canonique**.
5. Appel du module développé à l'étape 5 avec cet automate canonique pour lire un fichier cible et restituer numériquement sa table de symboles globale.

27 Difficultés Rencontrées et Solutions

1. **Corruption de la mémoire (`strtok`)** : Le C manie les chaînes avec sévérité, et `strtok` a causé une modification permanente (*mutation*) des variables passées en paramètres d'où l'importance de manipuler nos buffers localement via `char ligne[512]`.
2. **Pertes Mémoires** : Lors de l'enchaînement de la Partie 6, la saturation de la RAM (fuit mémoire) pointait alors que la mémoire des algorithmes intermédiaires non conservées (Thompson, DFA non minimal) était stockée. Nous avons encadré sérieusement cela en insérant une libération de mémoire immédiate (`free_automaton`) après chaque phase transitoire.

28 Conclusion de cette dernière phase

Les deux parties clôturent un travail laborieux par la concrétisation ultime du projet de compilation. L'architecture a prouvé une excellente robustesse pour soutenir un volume non anodin de lexèmes depuis un fichier source complexe jusqu'à un automate de reconnaissance optimal créé à la volée.

29 Conclusion Générale

Ce projet nous a permis de concrétiser les concepts abstraits de la théorie des langages en un outil logiciel robuste et polyvalent. De la simple lecture d'un fichier DOT à la transformation complexe d'expressions régulières en automates optimisés, nous avons couvert l'intégralité du pipeline de traitement. Cette expérience a renforcé nos compétences en programmation système (C), en gestion de versions (Git) et en travail d'équipe collaboratif.